# Classic Bluetooth Security and Exploitation

Rivera Jordan

*Digital Forensic and Cyber Security University at Albany State University of New York*

*Jrivear7@albany.edu*

Yuksel Celik

*Digital Forensic and Cyber Security University at Albany State University of New York*

*ycelik@albany.edu*

**Abstract: Bluetooth is a communication protocol people are familiar with when it comes to headphones, speakers, car kits, and wearable technology. On a technical level, Bluetooth is more complex and a very effective protocol to transmit data for specific services. Bluetooth, on the technical level, is quite difficult to understand, especially on the programming level, due to complicated documentation and private information. Private information restricts many programmers and other interested people from truly understanding Bluetooth on a programming level. This, however, prevents malicious developers from exploiting Bluetooth and manipulating the services it provides. The purpose of this project is to highlight what Bluetooth is, What Security procedures are in place with Bluetooth, Bluetooth Vulnerabilities, and implementation of a BlueSnarfing like attack.**

**Keywords: Bluetooth Security, Bluetooth Exploitation, Classic Bluetooth**

## 1 Introduction

Bluetooth is a protocol developed to transfer different data between devices. There are two known types of Bluetooth: Bluetooth Classic and Bluetooth Low Energy. Bluetooth Classic pertains to devices that need consistent higher data rate connections like headphones, car kits, and speakers. It is designed to handle higher output and requires more power to make up for it. Bluetooth Low Energy pertains to many Internet of Things (IoT) devices as well; however, it is used for connections that don't require a lot of energy. This helps devices run for longer periods of time. For this Document, we want to focus specifically on Bluetooth Classic with a minor focus on Bluetooth Low Energy. We want to highlight the security surrounding Bluetooth Classic in pairing procedures and in protocol functions. Understanding the Security aids in constructing an attack on a target's device through spoofing and unauthorized Bluetooth protocol functions

## 2 Literature Review:

### 2.1 Bluetooth Technologies:

*Table 1 Bluetooth technologies*

| Technology | Use Case |
|---|---|
| Bluetooth Classic | Transmit data using more energy and larger packets |
| Bluetooth Low Energy | Transmit data using lower energy output |
| Bluetooth Mesh | Substandard containing a network of Bluetooth Low Energy Devices |

Each Technology is based on the Hardware a device has. Most devices contain both Bluetooth Classic and Bluetooth Low Energy, while others may contain only one. If a

device contains only Bluetooth Low Energy specifications, then it will not be allowed to use Bluetooth Classic. Hardware limits each of these technologies.

### 2.2 Bluetooth Classic:

Bluetooth Classic is a 2.4GHz band operating in the 2400-2483.5 range. Bluetooth Classic is separated into two different specifications called Basic Rate and Enhanced Data Rate. Basic Rate is required for the classic protocol and sends packet header and access code. The Enhanced Data Rate is not required but is used in the packet. Bluetooth Classic Protocols are separated into 7 different groups known as Radio Frequency Communication (RFCOMM), Service Discovery Protocol (SDP), Link Management Protocol (LMP), Logical Link Control and Adaptation Layer Protocol (L2CAP), and the Host Controller Interface (HCI). All these protocols are integrated into the Bluetooth Stack, each with their own relation to each other. The Bluetooth Stack is used by every device that utilizes Bluetooth Classic Communication.

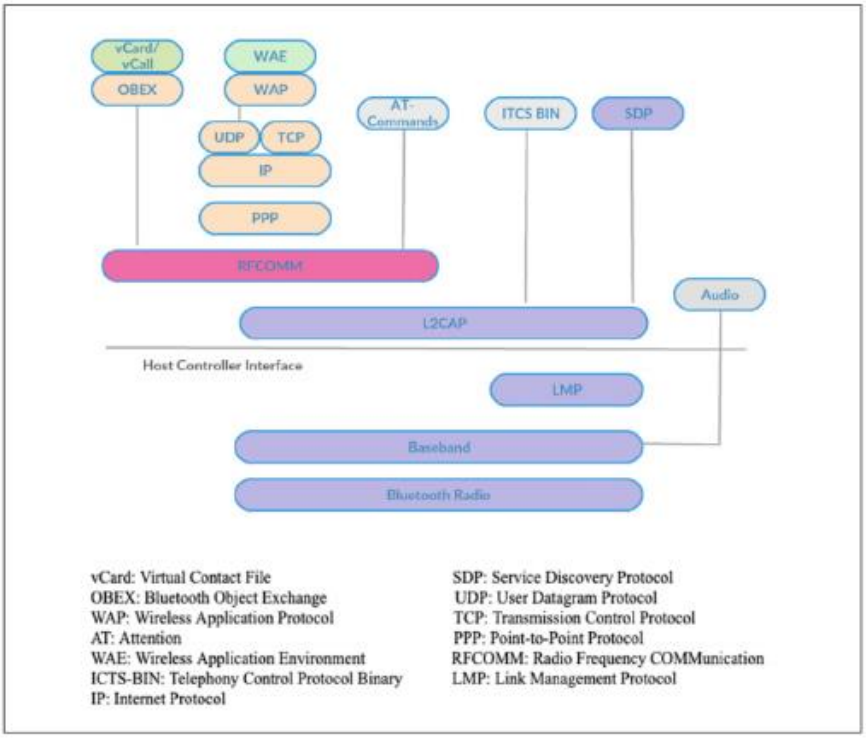



*Figure 1 Bluetooth Stack [1]*

The Bluetooth Stack is applied to devices like headsets, Car kits, keyboards, or other devices. Devices have higher energy output and more functions to use.

### 2.3 Bluetooth Classic Protocols:

Bluetooth Classic has an abundance of protocols it uses for various operations. For this project, we will be focusing on common

protocols that phones specifically use to carry out operations. These protocols are Message Access Protocol, Hands-Free Protocol, Phonebook Access Protocol, and Audio/Video Controller. Each of these Protocols has its own security implementations to prevent foreign access to a target device. For MAP, HFP, and PBAP, the User chooses to give a Bluetooth device access to these protocols, usually through a setting on the User's Device. Once these protocols are allowed on the device, the Bluetooth device is allowed to access contacts, phone calling, message control, and contact information known as Vcards. V cards contain information of a contact stored on the User's device [2]. Malicious or foreign access to these protocols can essentially control the important aspects of a phone. Each of these protocols have their own initiation and sub transmission protocols to handle their functions. For example, PBAP and MAP require using OBEX to access the Phone's storage and obtain contact information and message information. HFP and A/V use a series of AT commands to conduct operations on the devices.

### 2.3.1 Protocol Initiation:

These Protocols require an initiation phase to begin all the operations. The first stage of the initiation process begins with feeding in the User's MAC address into the Bluetooth device. It checks if was authenticated previously and if not requires the pairing process.

1. The second stage involves inputting the specific UUID for the protocol the User wants to utilize. Using the UUID, The port on which this service is provided can be found and inputted into code.
2. The Third stage is the verification of the RFCOMM connection. A connection between devices can be established using the Host MAC address, UUID, and Port. For this part, it is entirely dependent on what the Host Device is because that determines the pairing process between devices. If the connection is accepted, a confirmation Packet is sent else a termination is sent.
3. Once a connection is successful, data transmission is possible.
4. Connection is then terminated when disconnected

```
# Device Specific HFP UUID
uuid = "0000111f-0000-1000-8000-00805f9b34fb"


# Test for Parameters
service = bluetooth.find_service(uuid=uuid,address=addr)
print(service)
first = service[0]
print(first)
port = first["port"]
print(port)
name = first["name"]
print(name)
host = first["host"]
print(host)

# Establish a Bluetooth SOcket to connect to the Server
sock = bluetooth.BluetoothSocket(bluetooth.RFCOMM)
print(sock)
sock.connect((host,port))
print(bluetooth.lookup_name(addr))
```

*Figure 2 Protocol Initialization [3]*

## 2.4 Bluetooth Classic Pairing:

As mentioned before, Bluetooth Classic goal is to have data transmit between two devices through packet communication. To transmit data correctly to the desired devices, a pairing process is implemented to establish a connection with the devices. The pairing process requires a device transmitting a pairing request and a device receiving a pairing request. During this process authentication and encryption procedures are implemented to secure the connection between devices. Bluetooth Classic specifically uses a

symmetric Link key established through Legacy pairing and Simple Secure Pairing [3].

### 2.4.1 Legacy Pairing:

Legacy Pairing is a traditional pairing process involving a PIN (Personal Identification Number) of 16 bytes sent to the target Bluetooth MAC address as a generated key.

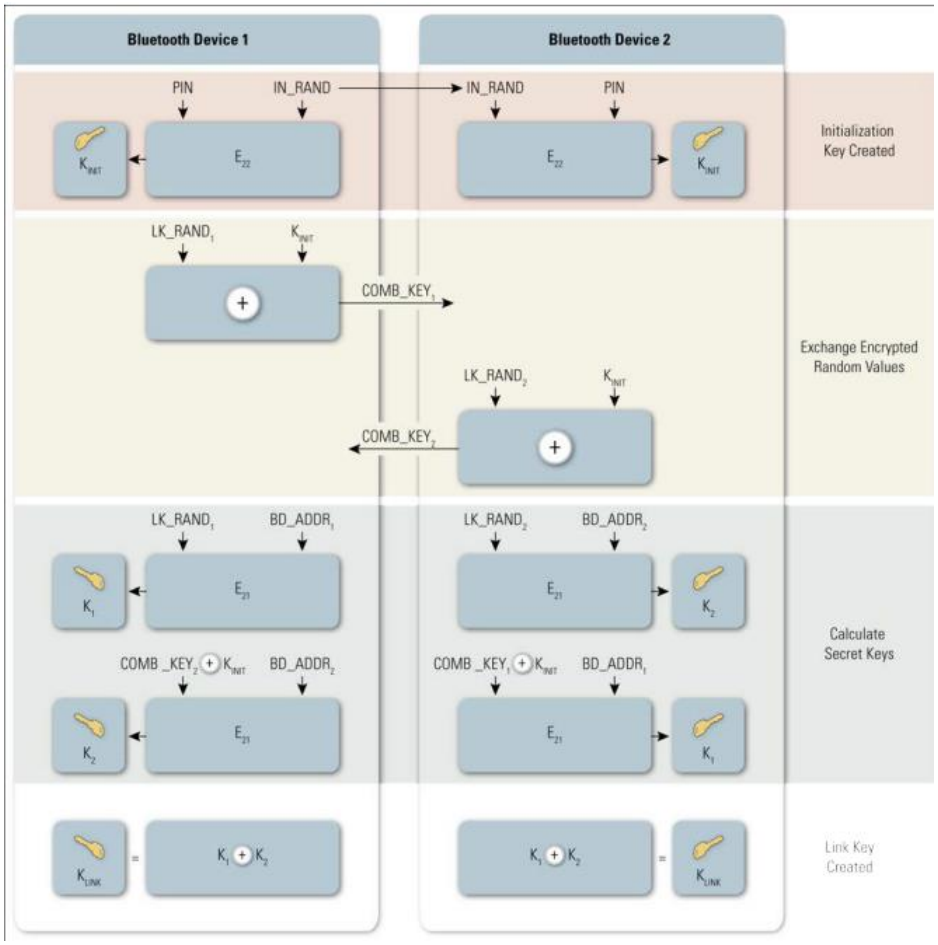



*Figure 3 Legacy PIN Pairing Process [3]*

The figure above shows the step-by-step process of Link Key generation through a random shared number and a single separately User known PIN. The Pairing Initiator (Device 1) creates that random number to be shared. Keys are created for each device and are exchanged to calculate their secret keys. Once this process is finished, the link key is created and stored on each device for each connection attempt. Without the corresponding Link Key, the device cannot be trusted in the Bluetooth Connection. Legacy Pairing is a lower security model used due to the weak number algorithm used in PIN generation [3].

### 2.4.2 Simple Secure Pairing:

Simple Secure Pairing (SSP) is another pairing process used in modern devices because of the higher security level. SSP uses a link key as well but is generated through asymmetric encryption. Now, SSP also has various sub-models that can be used for its pairing process.

*Table 2 Simple Secure Pairing [3]*

| Simple Secure Pairing Types | Description | Link Key Security Level |
|---|---|---|
| Numeric Comparison | Both Devices are given a number (not used for key generation) and a decision if they match. If they match, a link key is created, and the pairing process is successful | High |
| Passkey Entry | One device displays a number for the other device to enter. If the entered number matches, the pairing process is successful | Medium |
| Just Works | Allows a connection like numeric comparison. This connection is allowed without verifying the number | Low |
| Out of Band | Uses NFC (Near Field Communication) for pairing. Requires both acceptance for a successful connection | Medium |

Each type of SSP follows the same procedure and protocol for Key Generation. More specifically, SSP uses Elliptic-Curve Diffie-Hellman, which is an asymmetric encryption method [3]. Each of these keys can be either 256 or 192 depending on the device's specifications. Below is the Asymmetric Link Key Establishment.

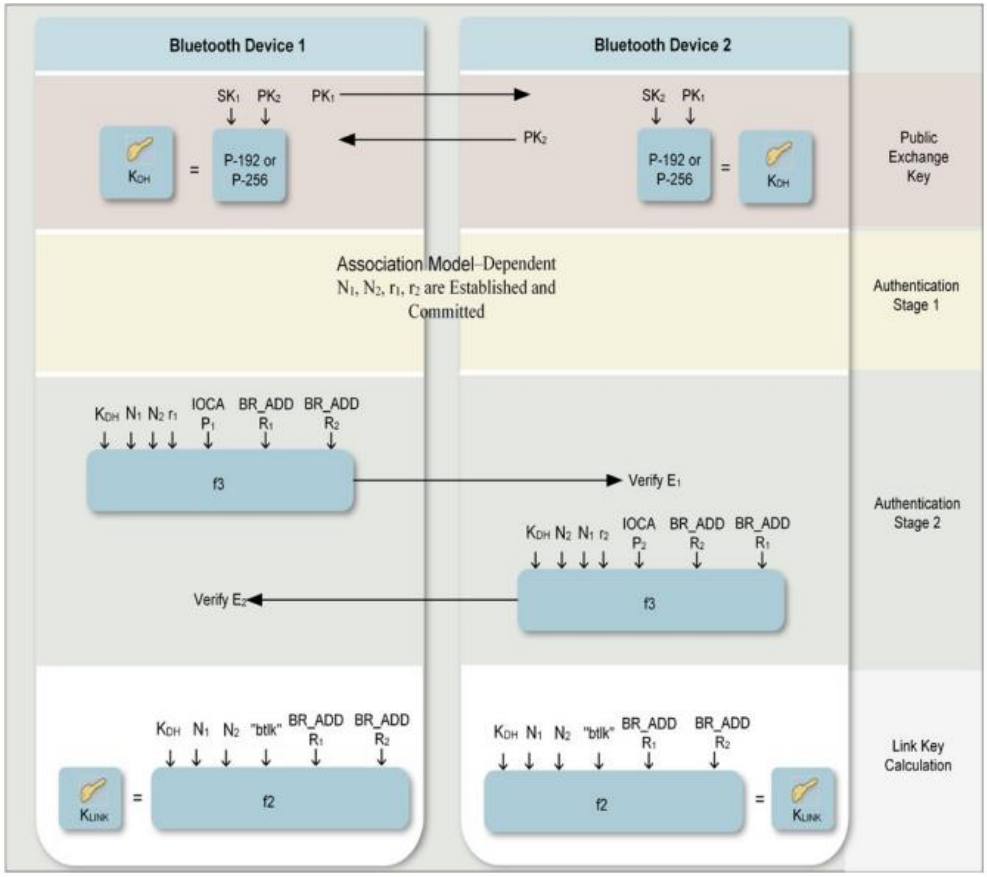



*Figure 4 Simple Secure Pairing Process (Diffie-Hellman) [3]*

The Diffie-Hellman implementation requires two keys (Public and Private). Each device exchanges its Public Keys to then proceed through authentication stages to verify each device's specifications. This includes both Bluetooth MAC Addresses and the values gained from the Association model. Once the Authentication stage is complete, the Link Keys are generated on each device to be stored for the next connection.

### 2.5 Bluetooth Classic Security:

Bluetooth security depends on two protocol sectors: the pairing process and the Data transmission process. Bluetooth Classic can suffer from various attacks based on the specific threat model the attacker uses.

*Table 3 Common Bluetooth Attacks*

| Type of Attack | Description | Awareness | Attack Result |
|---|---|---|---|
| BlueSnarfing | An Attack which gains unauthorized access to a Host's data including contact information and Messaging | An Attack which happens without the Host's Knowledge | Attacker will have all contact, Messaging, or other personal information accessible |

| | | | |
|---|---|---|---|
| | Information. | | through OBEX |
| BlueSmacking | An Attack which spams a Host device with packets to overwhelm the system to crash or create systemic damage. | This attack can happen without the User consent or knowledge. | The target device is rendered unusable or slow |
| BlueJacking | An Attack which involves sending SMS messages to other Enabled Host devices | This attack can happen without the User consent or knowledge. | The target is flooded with SMS messages without any defensive actions |
| Bluetooth Spoofing | An Attack which involves impersonating an already trusted device to connect to a Host | This attack can happen without the User consent or knowledge. | The target device is tricked into believing a foreign device is trusted and can lead to another attack |

These attacks cover both security sectors of Bluetooth Classic. Blue Smacking and Bluejacking do not require the pairing process to work. While Bluetooth Spoofing and BlueSnarfing attacks both require the pairing process and the Data Transmission process. These attacks do not require any physical infiltration and can be conducted virtually from a distance.

### 2.6 Bluetooth Low Energy:

Bluetooth Low Energy differs from Bluetooth Classic because it requires less energy and has less device functionality. Bluetooth Low Energy requires fewer channels of data transmission than Bluetooth Classic, which provides faster and more efficient packet speed. In comparison to Bluetooth Classic, Bluetooth Low Energy Stack is created with more efficiency and a simplistic design. The Bluetooth Low Energy Stack has 3 layers all consisting of different protocols in each. The First Layer is known as the Controller Layer, which contains the Link and Physical Layer. The Second Layer is known as the Host Layer, which contains the Logical Link Control and Adaptation Protocol (L2CAP), Attribute Protocol (ATT), Generic Attribute Profile (GATT), Security Manager (SM), and the Generic Access Profile (GAP). The Last Layer, known as the Application Layer, contains the user application. The Host Controller Interface interacts directly with the Link Layer and the L2CAP [4].

Bluetooth Low Energy Stack is mandatory for devices utilizing the Bluetooth Low Energy communication protocol. These devices consist of IoT devices specifically because they output lower amounts of energy.

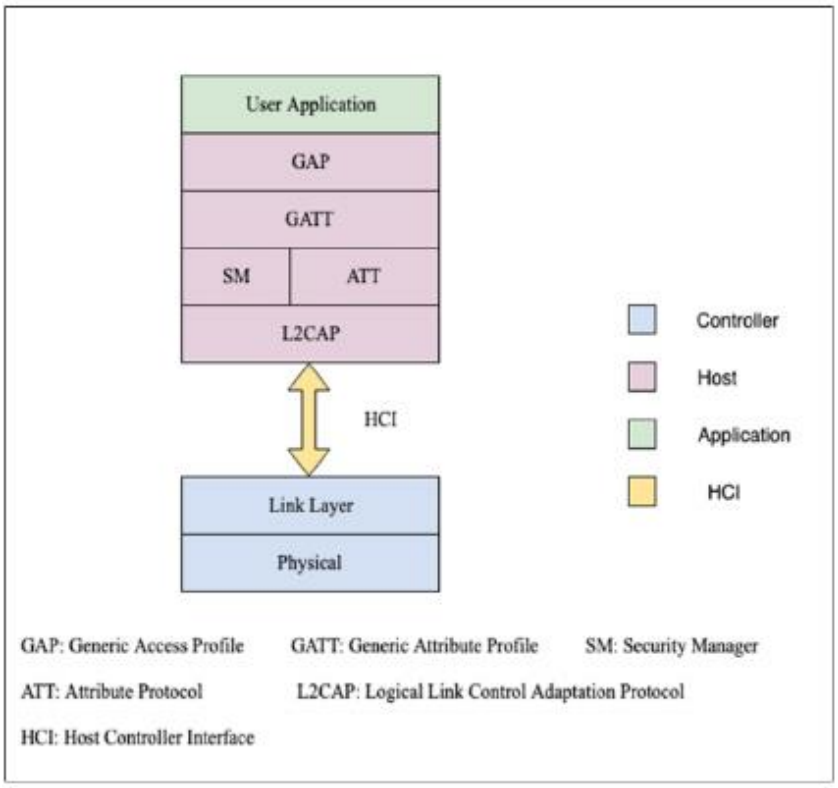



*Figure 5 Bluetooth Low Energy Stack [1]*

## 3 METHODOLOGY:

We separate our methodology into Device information extraction/Spoofing and information gathering. The information-gathering portion of our methodology is dependent on our spoofing implementation. The first part will show how to gain access to the target's device through Pairing Procedure sniffing and the second part is how to pull sensitive information without the User knowing using Bluetooth Classic Protocols. To gain access to the target's device, we need to infiltrate the pairing process between the two devices and access the Link Key between both. The Link Key will allow our device to spoof itself to trick the target's device into trusting the connection. Once access is gained, we can use the provided Bluetooth Classic Protocols or OBEX malicious code to pull Sensitive User Data without the User knowing. This implementation is done by spoofing a trusted car kit's pairing information containing Link Keys, Key information, and MAC Addresses because most of the features are enabled, bypassing the authorization process of those protocols. Unlike other Bluetooth devices, the target device has an extra layer of protection that prevents unauthorized use of provided protocols. The User needs to enable specific settings giving authorization to these protocols. If successful, an attacker can access contact information and message Information and have control over call functions.

### 3.1.1 Target Device Information Required:

- Bluetooth MAC Address
- Device Bluetooth Compatibility
- Device Range
- Type of Device
- Pairing Keys

When pairing devices, this information helps establish a trusted, well-documented connection. Device Bluetooth Compatibility is the most important information required. Since Bluetooth is split into two different subtypes with their own functions, it is essential to know if a device has the capability to the required type. This type of device is essential because it can give information on which functions are accessible and how to communicate effectively. There is a device range limit where it can affect the connection's stability. This can also determine which devices are located nearby. The Bluetooth MAC address serves as the Unique identifier for devices to identify each other. The Bluetooth MAC address is 48 bits long and can change depending on each device. The Pairing Keys are the security standard for the pairing process as described above. Pairing Keys, pairing encryption algorithms, and Key storage are usually implemented privately to prevent malicious actors from gaining easy access.

To conduct a Bluetooth Spoofing Attack, an attacker needs to conduct recon on Target's trusted device list or any devices nearby. For example, a pair of headphones or a specific car kit that is already paired or is being paired with Target's device should be identified. Once the MAC address of one of those devices is found, then the attacker can spoof their MAC address to act like the trusted device. This is the depreciated method of Spoofing a MAC address. [5] Our Implementation is like the figure above; however, it differs due to resource limitations. We will enable the HCI dump for Target's device, which is done from a physical connection or a USB sniffer. Once the HCI dump is available, we initiate a pairing procedure to extract the information from both devices. Once the information is gathered, we know what specifically the Target's device is looking for in a trusted connection. We take the information gathered and apply it to our Middleman device to be used in the spoofing portion. After the spoofing portion, we initiate another connection to extract the User's sensitive data through our bluesnarfing attack.

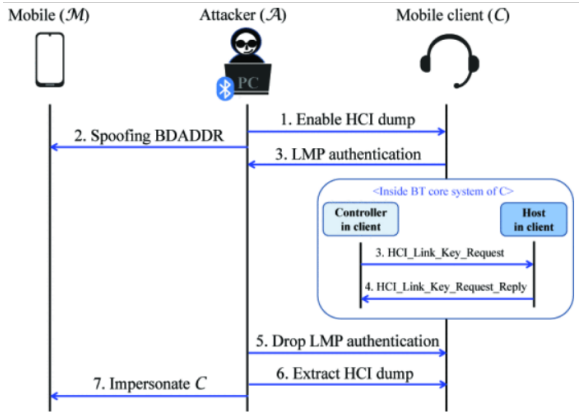



*Figure 6 Link Key Extraction and Spoofing [5]*

### 3.2 Link Key Implementation:

#### 3.2.1 Included Tools and Libraries:

- Raspberry Pi 4B 2GB
- Mac I5 13800k
- iPhone X Max
- Packet Logger
- Apple's Bluetooth Logs

For this part of the Methodology, an iPhone device is used to trace the Bluetooth activity with another device. Packet Logger is an Xcode Additional tool that allows developers to trace the Bluetooth Logs of an Apple device regardless of the protocol. This allows us to visualize the Bluetooth Classic communication and visualize the pairing procedure that occurs on a mobile device. To work effectively, the iPhone requires a Bluetooth configuration file provided by Apple, a Mac with Packet Logger installed, and another Bluetooth-enabled device to connect to. [6]

We begin our methodology by running the Bluetooth log file on the iPhone and

connecting it to the Mac. Once connected, the Packet Logger is opened and is configured to run iOS Trace. Through this iOS trace, we need to initiate a pairing sequence to observe the packet types, encryption protocol, and Key transmission. We conducted this through a test using a laptop and an iPhone where the Bluetooth Classic protocol is being used. Both devices are not paired with each other which helps showcase the real pairing implementation. To view the specific pairing packets, we filter it by looking for HCI Command packets and keywords related to Keys. We notice there are 2 keys generated: a Long-Term key for Bluetooth Low Energy and a Link Key for Bluetooth Classic. The Pairing Protocol used is the Just Works procedure where both devices display the same number and require both to be accepted. When pairing, a packet named "Link Key Notification" is received, providing the Bluetooth MAC Address of the Laptop, the Link Key, and the Key Type. Once this process is complete, the Phone stores the Link Key along with the associated Bluetooth MAC Address in a device manager.

We also want to test the pairing procedure after a device is remembered. Since the key generation has already been conducted, that means it is not required for the next time those devices pair. The only process the devices conduct is requesting and responding with a Link Key. A Link Key Request command awaited a reply from the requested device with a Link Key and the MAC Address. The Link Key Request Response Command is sent back containing the Link Key.

Now, since we see visually how pairing and key exchange work on a program level, we can try to test this on a known car kit. The car kit used for this implementation is the ENTUNE v 1.6 which is created by Toyota. ENTUNE is equipped with all Bluetooth Classic protocols and utilizes a Just Works pairing procedure. Once the Link key is retrieved from the HCI dump, we can then alter the Raspberry Pi's Bluetooth MAC address and Link key to match the Car kit. We need the three fields: Key Type, Link Key, and MAC address to be spoofed.

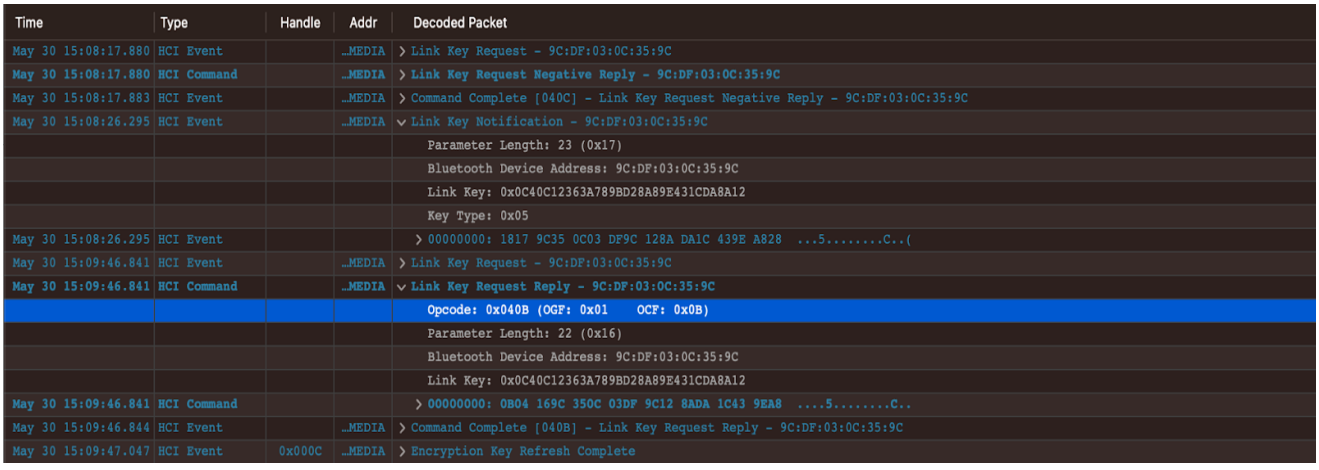



*Figure 7 Packet Logger ENTUNE Link Key*

Before we connect to the iPhone, we first need to ensure the Raspberry pi and the iPhone are not previously paired. If not paired, we proceed to the pairing step and shouldn't be prompted to a new pairing procedure. The

iPhone should “remember” the Raspberry Pi as the car kit it previously paired with.

### 3.3 BlueSnarfing Implementation:

#### 3.3.1 Included Tools and Libraries:

- Raspberry Pi 4B 2GB
- Python 3.1 or Higher
- PyBluez
- PyOBEX

The Raspberry Pi will serve as the attacker’s device and be used as a Spoofed Bluetooth device. The Raspberry Pi is running RaspOS, which is an Ubuntu-based Operating System. This is important to take note of because Ubuntu has different procedures for Bluetooth implementation and procedures in comparison to Mac OS and Windows OS. It is crucial this Raspberry Pi or any other device used in this attack has Bluetooth Classic capabilities. The Exploit Code is written in Python utilizing two important external libraries known as PyBluez and PyOBEX. PyBluez is the Bluetooth Library that Linux compatible and serves as the method of finding Bluetooth-enabled devices, Gathering Information, establishing a connection, and sending/receiving commands. PyOBEX is a library that aids with working with OBEX in Python. OBEX is a complicated and hard-to-read protocol that contains files and information on the Host’s device. PyOBEX aids in reading pulled contact information and parses it to be readable.

#### 3.3.2 Included Protocols:

-Hands-Free Profile (HFP) [6]

-Message Access Profile (MAP) [7]

-PhoneBook Access Profile (PBAP) [8]

Once the devices are effectively paired, we must establish separate protocol connections to conduct an attack. To Conduct a BlueSnarfing Attack, these three protocols are utilized to pull contact information in the form of Virtual Cards. These Virtual Cards contain Phone Numbers, Emails, contact names, Contact Images, and address information. The contained content is entirely dependent on which device holds the VCards. For example, iOS devices based Vcards may have different information in comparison to Android devices. Attackers can request specific Vcards using OBEX which can aid in their attack. Hands-Free Profile can also be used to pull contact information using Attention Commands (AT commands). Attackers can abuse AT commands to specifically control calling functions, control phonebooks, and even gather device information. iOS devices may have different AT commands hidden from the public to enhance their security. Attackers can try out variations of the same AT commands to carry out iOS-based attacks. [7]

Each of these protocols has its own UUIDs and their own ports (Depending on the Device) to help establish specific data transmission.

*Table 4 Protocol Details [8, 6, 7]*

| Protocol | UUID |
|---|---|
| Hands-Free Profile (HFP) | 0000111f-0000-1000-8000-00805f9b34fb |
| Message Access Profile (MAP) | 00001132-0000-1000-8000-00805f9b34fb |
| PhoneBook Access Profile (PBAP) | 796135f0-f0c5-11d8-0966-0800200c9a66 |

Utilizing each UUID and Port we can now ask the Target’s Device to access these protocols. Once a request is sent, protocol functions can be used if the User has enabled them in the past.

For this implementation, we will use PBAP to pull contacts and their data associated with

them. Below is the code for the PBAP implementation. [8]

```
59      def pull_missed_calls(self):
60          missedCalls = self.obex_session.get("telecom/mch",header_list=[Type(b'x-bt/vcard-listing')])
61          print(missedCalls)
62
63      def pull_incoming_calls(self):
64          incomingCalls = self.obex_session.get("telecom/ich",header_list=[Type(b'x-bt/vcard-listing')])
65          print(incomingCalls)
66      def outgoing_calls(self):
67          outgoingCalls = self.obex_session.get("telecom/och",header_list=[Type(b'x-bt/vcard-listing')])
68          print(outgoingCalls)
69      def mainPhoneBook(self):
70          Contacts = self.obex_session.get("telecom/pb",header_list=[Type(b'x-bt/vcard-listing')])
71          contactsList=parse(Contacts[1])
72          self.obex_session.setpath("telecom/pb")
```

*Figure 8 PBAP Code Functions*

```
86      def vcardEntry(self,info):
87          self.obex_session.setpath("telecom/pb")
88          card = self.obex_session.get(name=str(inf
89          try:
90              data = card[1]
91              vcard_string = data.decode('utf-8')
92              vcard =vobject.readOne(vcard_string)
93              vcard.prettyPrint()
94
95              return vcard
```

*Figure 9 PBAP V card Pull Function*

Another protocol we will use is the HFP protocol to initiate a phone call to a specific number.

```
32      # This process is manadatory for func
33      # This is the step by step intializat
34      command = "AT+BRSF;\r\n"
35      sock.send(command.encode())
36      response = sock.recv(1024)
37      print(response.decode())
38      command = "AT+CIND=?\r\n"
39      sock.send(command.encode())
40      response = sock.recv(1024)
41      print(response.decode())
42      command = "AT+CMER=\r\n"
43      sock.send(command.encode())
44      response = sock.recv(1024)
45      print(response.decode())
46      command = "AT+CHLD=?\r\n"
47      sock.send(command.encode())
48      response = sock.recv(1024)
49      print(response.decode())
50
51
52      # Everything after this can be any co
53      command = "ATD<PhoneNUmber>;\r\n"
54      sock.send(command.encode())
55      response = sock.recv(1024)
56      print(response.decode())
```

*Figure 10 HFP Implementation [7]*

The code above displays the step by step process to initiate a call using HFP. The logic of this code is to send specific AT commands to the Target device and ensure there is a response for

each command. The final command that is sent on Line 53 is the AT command to start a call. Between <>, the Attacker can input any phone number they want, and the device will initiate to that number.

## 4 SOLUTIONS:

The best solution for a user to avoid these attacks would be to turn off Bluetooth entirely. However, this can be useless when a User requires Bluetooth for an activity. Having Bluetooth turned off denies any connection, whether legitimate or malicious, from occurring. For a more reasonable approach, it is entirely dependent on the attack that is conducted.

If it is a Spoofing attack, we want to minimize the opportunity of accessing the Link Key, MAC address, and any pairing procedure the device has. This can be done by restricting the HCI dump access and encrypting Link Key packets more efficiently. Link Key exchange is easily viewed in the HCI dump in plaintext, allowing Attackers to copy them with no effort. The Link Key packets contain all the required information for the attacker to leverage in their attack. Restricting HCI dumps to be accessed by reputable individuals and restricting some functionality can help prevent these spoofing attacks. Bluetooth MAC Addresses are accessible and should be hidden from the public view. Some Bluetooth Devices can construct new Link Keys to be used in their Pairing procedures preventing the same key from being stored or viewed after each initialization. [5]

Solutions for the BlueSnarfing attack include turning off Bluetooth when it is suspected your device has been connected without your permission. Most of the time, Bluesnarfing attacks can go unnoticed, making it exceptionally difficult to prevent. Increased Awareness by analyzing suspicious behavior like weird Messages, buggy Phone calls, contact changes, etc. For developers, solutions include adding more authorization methods to help layer security for these protocols. Modern Devices have implemented Enable-Disable features for these protocols, giving users the ability to control what a connected device does.

## 5 RESULTS:

### Spoofing Results:

The Spoofing results worked as well given we successfully extracted required sensitive pairing information. We were able to obtain the Link Key, Key Type, and the MAC address to be used to track the target’s device. Challenges that occurred included applying changes to the Raspberry Pi to act as the Middle Man. The information was changed and worked on one remembered pair, but it would not work if repeated. This may be due to Apple implementing Hidden security protocols or extra information withheld in the HCI Dump. It is theorized Apple may include device speciation in the pairing procedure to remember in an event a device is spoofed. It is also theorized Apple may also regenerate new keys based on the Link key communication in Figure 8.

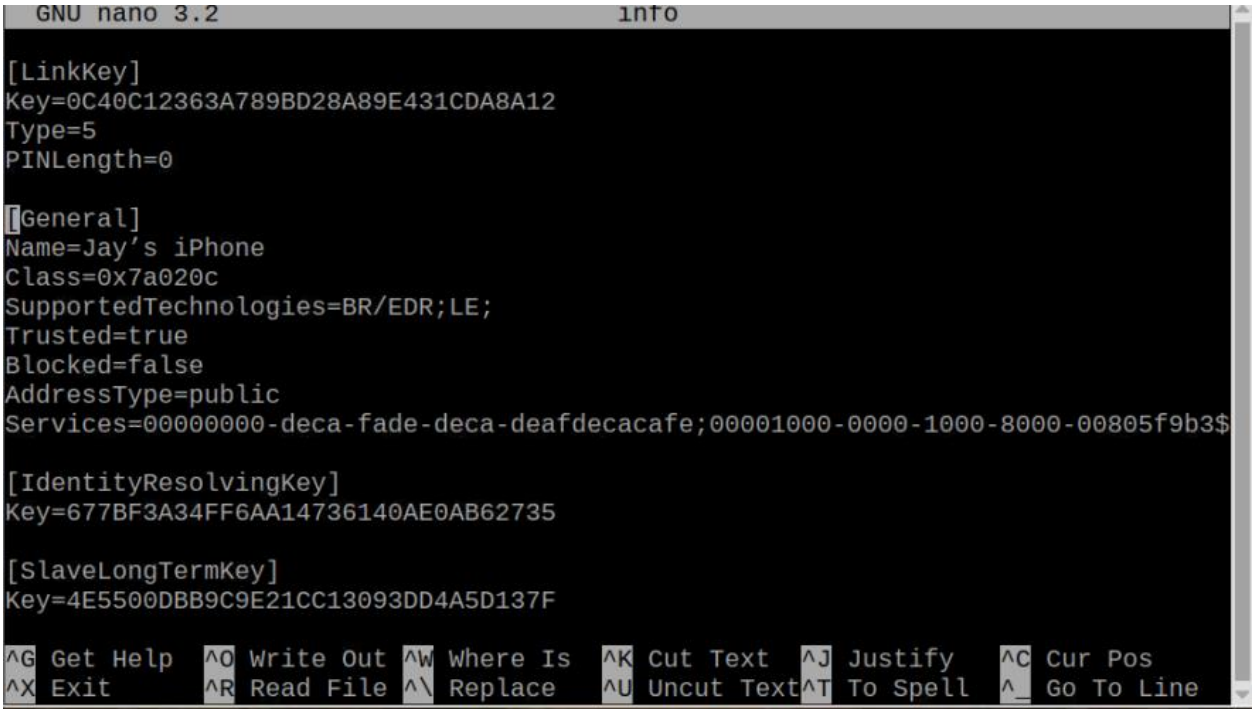



*Figure 11 Raspberry Pi Bluetooth Information*

Figure 11 showcases the details of the target's device. A new section is the identification key and the Slave Long Term Key. The Slave Long Term Key and the Identification Key refers to the Bluetooth Low Energy protocol. The Link Key is edited here to mimic the Link Key obtained in Figure 8. After the initial connection the device is then forbidden

**BlueSnarfing Results:**

The Bluesnarfing Code worked efficiently to where the Phonebook was accessible to the attacker. All Metadata, including Email, Phone Numbers, Contact Names, Contact Pictures, and Addresses, were included in the attack. The User is not made known of the attack only when PBAP was incorporated. For the HFP implementation, the code worked to where a call is placed, however this was limited due to the extent the code needs an Audio portion to listen into a call. This also provides the problem of the User being known of a call being placed on their device. The HFP code still needs to be refined to give more control to the attacker, but the establishment of the protocol worked effectively. The MAP protocol did not work for this implementation because iOS may have an extra protocol or a different UUID for this protocol. When initiating the connection, it returns Message packets with no results, making this protocol attempt not feasible for now.

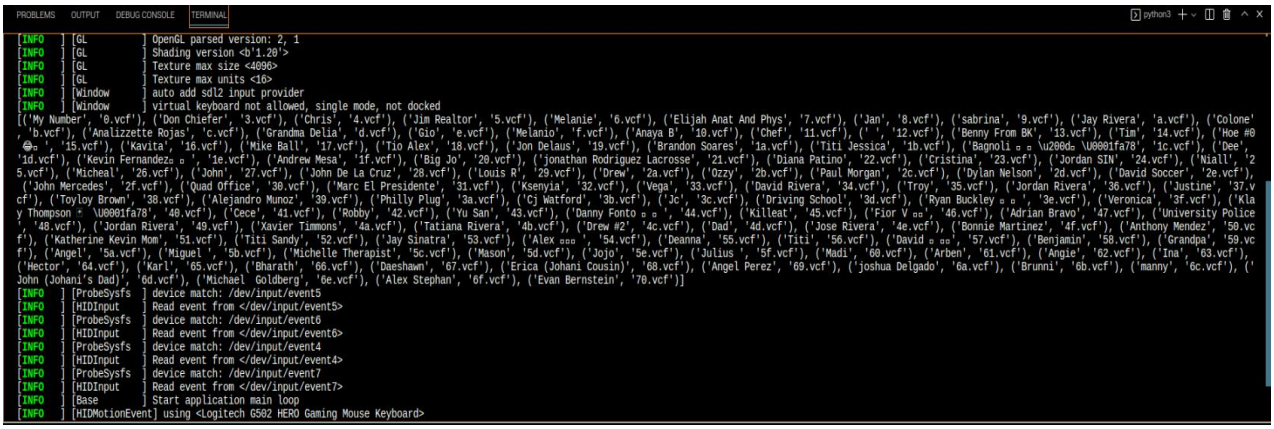



*Figure 12 PBAP Vcards from Target's Device*

Figure 11 displays the list of contacts in the form of Vcards. This list dynamically changes and has an extension .vcf

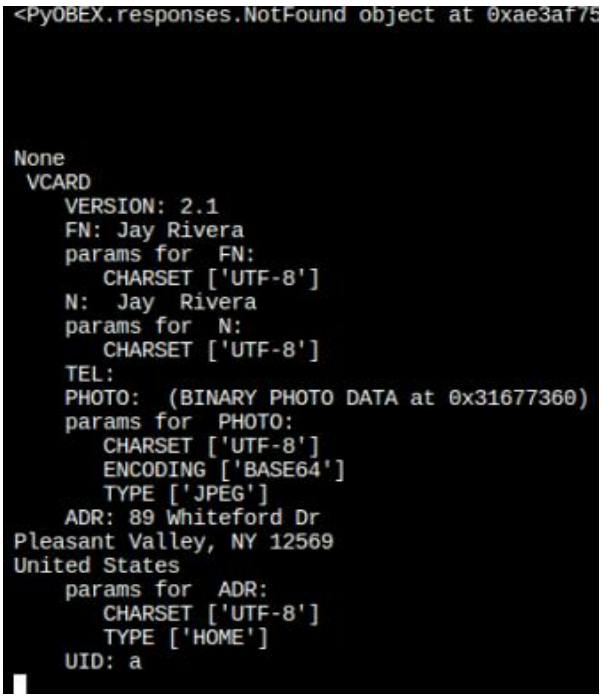



*Figure 13 VCard Parsing*

Vcards contents are then parsed to display all the Meta Data to be readable in Figure 12

## 6 CONCLUSION:

As discussed, Bluetooth Classic is implemented in almost every mobile device, allowing it to have extended functionality. We discussed the multiple pairing procedures to ensure connections are trusted, the information transmitted between trusted devices during the pairing procedures, and the protocols associated with accessing User sensitive information. Our Methodology highlights the importance of the pairing process packets and the contents, allowing attackers to spoof devices and bypass User authorization features within their mobile device. Although we didn't effectively spoof the device like a real attack, we understand the process and required contents to spoof a device to be trusted. We do intend to continue our research and understanding of Bluetooth to hopefully achieve the goal of spoofing Car Kit pairing information more effectively without an HCI dump.

## 7 REFERENCES


[1] P. P.-R. M. H.-B. (. C. L. Jesus Alvarez-Cedillo, "Bluetooth Intrustion Techniques," 2012.

[2] C. Gomez, J. Oller and J. Paradells, "Overview and Evaluation of Bluetooth Low Energy; An Emerging Low-Power Wireless Technology," 29 08 2012.

[3] N. U. D. o. Commerce, "Guide to Bluetooth Security," 05 2017.

[4] T. P. J. S. G. T. Matthias Cäsar, "A survey on Bluetooth Low Energy security and privacy," *Computer Networks*, vol. 205, no. 108712, 2022.

[5] J. K. a. J. H. C. Koh, "BLAP: Bluetooth Link Key Extraction and Page Blocking Attacks," in *52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, Baltimore, 2022.

[6] T. a. A. W. G. Bluetooth SIG Proprietary Audio, Hands-Free Profile, 2020.

[7] T. a. A. W. G. Bluetooth SIG Proprietary Audio, Message Access Profile, 2019.

[8] T. a. A. W. G. Bluetooth SIG Proprietary Audio, Phonebook Access, 2019.

[9] Apple, "Bluetooth Profiles that iOS and IPadOS support," 31 10 2023.

[10] C. Hensler and P. Tague, "Using Bluetooth Low Energy Spoofing to dispute device details," 2019.

[11] M. Stute , A. Heinrich, J. Lorenz and M. Hollick, "Disrupting Continuity of Apple's Wireless EcoSystem Security: New Tracking, DoS and MitM Attacks on iOS and mac OS through Bluetoth Low Energy,AWDL and Wi-fi," 13 08 2021.

[12] F. L., J. Kühl and F. Papí, "Bluetooth Low Energy Urban Networks for Ultra-Accurate GeoLocation Applications," in *Transport Research Arena (TRA) Conference*, 2023.

[13] T. C. a. J. L. Esteves, "BlueMirror: Reflections on Bluetooth Pairing and Provisioning Protocols," pp. 339-351, 2021.

[14] M. S. C. J. J. Y. Prathibha Muraleedhara, "Any Bluetooth device can be hacked. Know how?," *Cyber Security and Applications*, vol. 2, no. 100041, 2024.

[15] S. D. B. M. S. H. M. A. Shaikh Shahriar Hassan, "Security Threats in Bluetooth Technology," *Computers & Security*, vol. 74, no. 0167-4048, pp. Pages 308-322, 2018.

[16] S. C. (. B. (. S. (. A. K. (. A. Matheus E. Garbelini (SUTD), "BRAKTOOTH: Causing Havoc on Bluetooth Link Manager," NRF NATIONAL SATELLITE OF EXCELLENCE IN TRUSTWORTHY SOFTWARE SYSTEMS, 2021.